

Design Document for openJuice

mikomikotaishi

General guidelines

Use the latest stable release of C++ (at the time of writing, that is C++23).

Use external libraries sparingly - and only when absolutely necessary (i.e. `<regex>` uses an inferior regular expression implementation, so opt to use `boost.regex`, or `boost.asio` for networking capabilities).

Build scripts

There are scripts that are used to compile the project simply:

- `quick-cmake-build.py` (Python; for simplicity and cross-compatibility)
- `QuickCMakeBuild.java` (Java with JBang; experimental)

Discord SDK Update Scripts

This directory contains scripts to automatically update Discord SDK files from the `discord_social_sdk` directory to the `lib/discord` directory.

Usage

Method 1: Python Script (Recommended)

`./update_discord_sdk.py`

Method 2: Bash Script

`./update_discord_sdk.sh`

How it works

When you drag and drop a new `discord_social_sdk` folder into the project root, run either script to automatically:

1. Copy all platform-specific libraries to the correct locations
2. Update the licence file
3. Handle the iOS framework directory structure
4. Provide detailed logging of all operations

File Mappings

The scripts copy files according to this structure:

```
discord_social_sdk/      → lib/discord/
├─ License-Notices.txt   → License-Notices.txt
├─ lib/release/
│   ├─ discord_partner_sdk.aar      → android/discord_partner_sdk.aar
│   ├─ libdiscord_partner_sdk.dylib → darwin/libdiscord_partner_sdk.dylib
│   ├─ libdiscord_partner_sdk.so    → linux/libdiscord_partner_sdk.so
│   ├─ discord_partner_sdk.lib      → win32/discord_partner_sdk.lib
│   └─ discord_partner_sdk.xcframework → ios/discord_partner_sdk.xcframework
└─ bin/release/
    └─ discord_partner_sdk.dll      → win32/discord_partner_sdk.dll
```

Requirements

- Python 3.6+ (for Python script)

- Bash (for shell script)
- The `discord_social_sdk` directory must exist in the project root

Both scripts will create the necessary directory structure automatically.

Coding style

When contributing, make sure that you have `git pulled` the most recent revision.

Import statements and include directives

Generally we will avoid writing `#include` as much as possible, however it is certainly impossible to completely forgo the use of this directive. Use `#include` sparingly, only when absolutely necessary. Some cases in which it is necessary include writing `#include <filesystem>` to allow aliases from `std.fs` to work, or using `#include <toml++/toml.hpp>` or `#include <SDL3/SDL.h>` In all other cases, simply invoke `import` instead.

With all import statements, we aim to match the Java style for organising the use of `import`. This means:

- `import` all (homemade) standard library modules (under `std` first), alphabetically.
 - The exception to this rule is the module `std.core`, which is always `imported` first.
- `import` all module partitions (if any)
- `import` all internal modules necessary, alphabetically
- `import` all external modules necessary, alphabetically (if any)

For standard library modules, while you are free to `import` only the needed header within the module (i.e. `import std.fs.filesystem;` instead of `import std.fs;`), please do so only sparingly and only if you have a very compelling reason.

Namespaces

If using all symbols from an imported namespace, simply use `using namespace` to reduce cluttering the codebase. For instance, whenever anything of the `std::collections` namespace is invoked, we write `using namespace std::collections;`

However, as mentioned with regards to importing symbols from namespaces, there are exceptions to when to use `using namespace` and when to only write `using`. For instance, whenever invoking a smart pointer from the namespace `mem`, we prefer to write, for instance, `using std::mem::SharedPointer, std::mem::UniquePointer;` (for whichever smart pointers are needed), in an alphabetised list.

Warnings

If at any point outdated or deprecated parts of the standard library must be used, mark the area using the preprocessor directive `#warning`, followed by a descriptive and consistent warning of the issue. The `#warning` directive will display a compiler warning during compilation so that this issue can be found again and handled some time in the future.

Doxygen comments

Use Javadoc-style comments. Comment all classes, methods, fields, enums, namespaces, namespace members, and important global variables/constants. Obviously, this does not mean to write copious amounts of documentation where it is unnecessary - so long as comments something is not obvious, it should be commented.

Classes and class relations (Inheritance)

Indicate any important details about a class (such as its purpose, whether it is singleton, etc.) Specify whether a class is concrete, or if it is abstract. See below on guidelines:

Use `@class` to denote a concrete class, a utility class (in the sense of Java), or an abstract class that has fields. Use `@interface` to denote an abstract class without fields (an “interface”). (Even though an interface, strictly speaking, does not have constructors, we will continue to count an abstract class with only methods as an interface.) An interface should be prefixed with `I` to indicate that it is an interface (as C++ does not allow for a separation of `extends` and `implements`).

Use `@extends` to indicate that a concrete class inherits from another concrete class, or to indicate that an abstract class (an “interface”) inherits from another abstract class. Use `@implements` to indicate that a concrete class inherits from an abstract class.

Refer to any class that is abstract (or should be abstract) as an “interface” in Doxygen comments (although C++ does not offer “interfaces” in the strictest sense, this is done for clarity). Only refer to a concrete class as a “class” in Doxygen comments.

When applicable, create a “marker interface” with no methods (besides a `protected` constructor and `public virtual` destructor) that is used solely to denote that a class implements some behaviour. Such marker interfaces belong in `openjuice.engine.util.Interfaces:*`.

Namespaces

Indicate the purpose of a namespace.

Avoid using utility classes (in the sense that they exist in Java - a static class with static methods with a private constructor) in favour of using namespaces, unless there is a truly compelling reason to use a utility class.

Standard library modules

Indicate the header that the standard library module is associated with, and indicate the purpose of the grouping of standard library header modules.

Module naming scheme

This project follows a Java-style convention of naming modules, with the primary difference of omitting the top level domain and project name. In other words, the module should be named by its path relative to the project root delimited by `.` (to denote hierarchy), with the final word in the module name being the name of the file, in PascalCase. The name of the file itself should be in PascalCase.